🟦 Day 39 – Memento Design Pattern

> **Memento Design Pattern** is used when we want to **save snapshots of an object's state** so that we can later **restore the object to a previous state**.

🟩 What is Memento Design Pattern?

Memento is a **Behavioral Design Pattern**.

Its main purpose is:

```
Save Object State
       ↓
Create Snapshot
       ↓
Object changes
       ↓
If required → Restore old state
```

Simple Example

Suppose an object's state keeps changing:

```
Object
  │
  ├── State 1
  │
  ├── State 2
  │
  └── State 3
```

Now suppose we want to save the object whenever it reaches a new state.

```
State 1 → Snapshot
State 2 → Snapshot
State 3 → Snapshot
```

If later something goes wrong, we can go back:

```
State 3
  ↓
Rollback
  ↓
State 2
```

So Memento mainly provides:

- **Save object's state**
- **Snapshot**
- **Undo capability**
- **Rollback to a previous stable state**

🟩 Why Do We Need Memento?

Suppose we have an object whose internal state changes continuously.

```
        State changes
              ↓
Object → State 1 → State 2 → State 3
```

We may need the **previous state** later.

But if we directly change the object's internal data, the old state can be lost.

Therefore:

```
Current State
     ↓
Take Snapshot
     ↓
Change State
     ↓
If something goes wrong
     ↓
Restore Snapshot
```

Main Idea

> **Memento stores a snapshot of the object's state so that the object can be restored later.**

🟩 Three Magic Words of Memento ⭐

Memento Pattern has **3 important components**:

```
1. Originator
2. Memento
3. Caretaker
```

🟧 1. Originator

What is Originator?

Originator is the **object whose state keeps changing**.

Example:

```
Database
Game
Text Editor
Document
```

Its responsibilities:

- Its state changes.
- Creates Memento.
- Restores state from Memento.

In short:

```
Originator
    │
    ├── createMemento()
    │
    └── restoreMemento()
```

The notes give an example of a database as the Originator.

🟧 2. Memento

Memento is basically a **snapshot of the Originator's state**.

Its responsibilities:

- Stores Originator's internal state.
- Provides the saved state when restoration is required.

```
Originator
    │
```

```
    | creates
    ↓
 Memento
    |
    └── stores previous state
```

Example:

```
Database current state:

rollNo = 101
name   = Aditya
```

Memento stores:

```
rollNo = 101
name   = Aditya
```

Later this snapshot can be used for rollback.

🟧 3. Caretaker

Caretaker manages the Mementos.

Responsibilities:

- Manages Mementos.
- Stores a **list/history of Mementos**.
- Initiates operations such as **begin / commit / rollback**.

```
Caretaker
    |
    ├── Memento 1
    ├── Memento 2
    ├── Memento 3
    └── Memento 4
```

Important:

> **Caretaker manages the snapshots but does not modify the actual state stored inside the Memento.**

🟩 Easy Relationship

Remember this:

```
        creates
Originator ──────────► Memento
                          ▲
                          │
                          │ stores/manages
                          │
                     Caretaker
```

Or:

```
Originator
   │
   │ create snapshot
   ▼
Memento
   ▲
   │
   │ manages
Caretaker
```

Easy Trick

```
Originator → State ka owner

Memento → State ka snapshot

Caretaker → Snapshots ka manager
```

🟩 Database Transaction Example

The notes use **Database Transaction Manager** as the main example.

Suppose database mein hum kuch values change kar rahe hain.

For example:

```
Before:

rollNo = 101
name   = Aditya
```

Now transaction starts.

We want to change:

```
rollNo → 102
name   → New Name
```

But suppose some issue occurs.

Only:

```
rollNo → 102
```

gets updated.

But:

```
name
```

doesn't get updated.

Now database is in an **inconsistent state**.

```
rollNo = 102
name   = Old Name
```

This is dangerous because the transaction was only partially completed.

🟩 6. Database Inconsistency

Database inconsistency can happen because of:

1️⃣ Failed CRUD Operation

```
Create
Read
Update
Delete
```

If an operation fails halfway, the database may become inconsistent.

2️⃣ Partial Updates

Some values get updated while others don't.

```
Expected:

A → Updated
B → Updated
C → Updated

Actual:

A → Updated
B → Not Updated
C → Not Updated
```

This creates an inconsistent state.

🟩 Solution → Rollback

Solution:

```
Rollback
```

Rollback means:

> **Revert the database back to its previous consistent state.**

Flow:

```
Previous Consistent State
          |
          | Snapshot
          ▼
       Memento
          |
          ▼
   Transaction Starts
          |
          ▼
    Database Changes
          |
       ┌──┴──┐
       |     |
    Success  Failure
       |       |
```

```
      ▼      ▼
   Commit   Rollback
              │
              ▼
        Previous State
```

Database Transaction Flow

The notes show the following transaction flow:

```
Transaction
     │
     ├── calls Originator
     │
     └── stores Memento
```

Then:

```
Transaction
     │
     ▼
Database Changes
     │
     ├── Everything fine?
     │
   YES│          NO
     ↓            ↓
  Commit       Rollback
                  │
                  ↓
          Previous State
```

If transaction succeeds

```
Transaction
    ↓
Everything fine
    ↓
Commit
    ↓
Delete stored Memento
```

The snapshot is no longer required after successful commit.

If transaction fails

```
Transaction
    ↓
Failure
    ↓
Rollback
    ↓
Restore previous state
```

🟩 Begin → Commit → Rollback

These three terms are important for understanding the example.

`begin`

Transaction start karta hai.

```
beginTransaction()
```

At this point, we can create/save a snapshot.

`commit`

If everything succeeds:

```
commit()
```

Means:

> Changes are successful. Keep them permanently.

The old snapshot can then be removed.

`rollback`

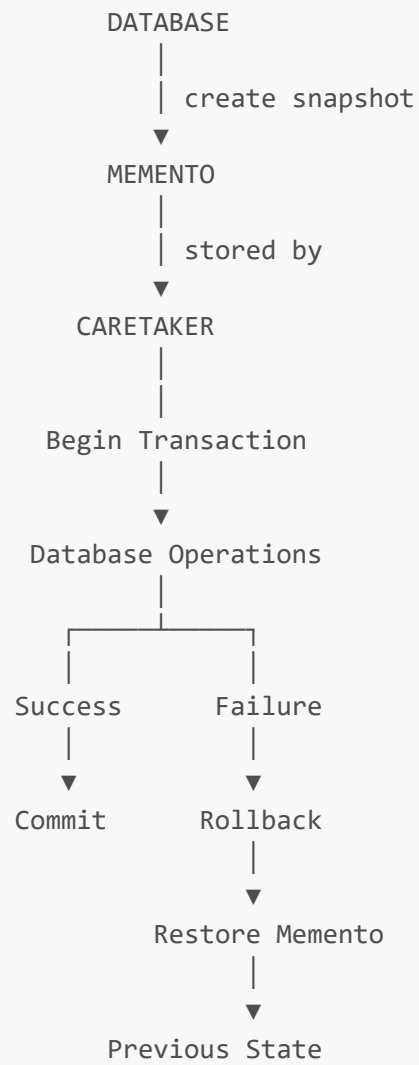
If something fails:

```
rollback()
```

Means:

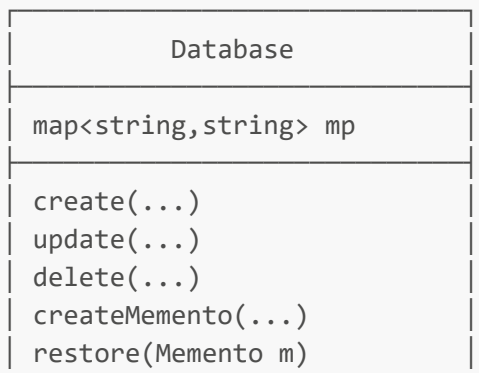
> Undo the changes and restore the previous state.

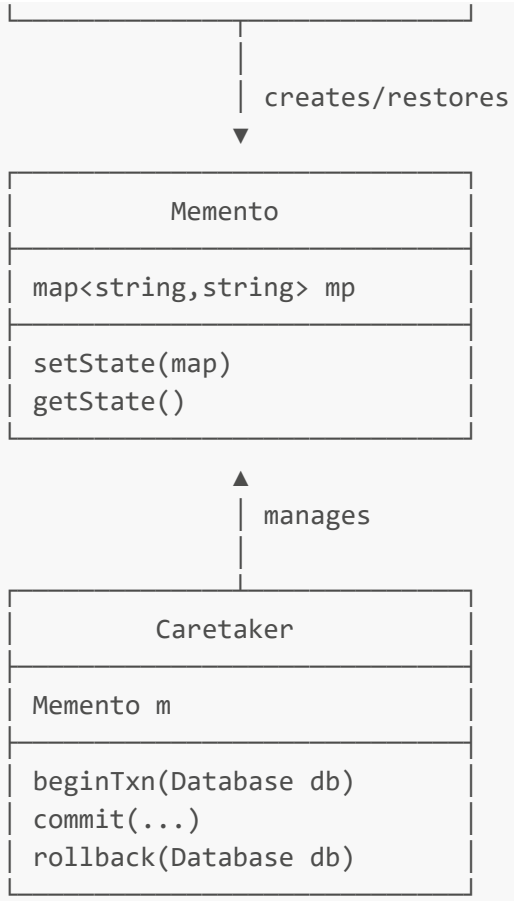
Complete Database Example



This is the core Memento flow from the notes.

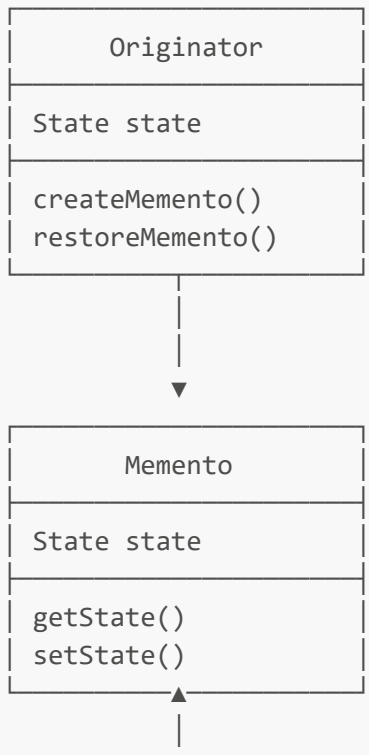
11. UML Diagram – Database Example





12. Standard Memento UML

The standard diagram in your notes has:



```
          |
┌─────────┴──────────┐
|      Caretaker     |
├────────────────────┤
| list<Memento> history|
├────────────────────┤
| add(Originator)    |
| undo()             |
└────────────────────┘
```

Standard UML shows the **Originator and Memento working with the saved State**, while Caretaker maintains the Memento history.

🟩 13. Standard Memento Flow

```
          Originator
              |
        createMemento()
              |
              ▼
           Memento
              |
              |
              ▼
         Caretaker
              |
        stores history
              |
              ▼
         [M1, M2, M3]
```

For undo:

```
Caretaker
    |
    | undo()
    ▼
Previous Memento
    |
    ▼
Originator.restoreMemento()
    |
    ▼
Previous State
```

🟩 Why Memento?

Without Memento:

```
Object
  ↓
State changes
  ↓
Old state lost ❌
```

With Memento:

```
Object
  ↓
Create Snapshot
  ↓
State changes
  ↓
Problem?
  ↓
Restore Snapshot
  ↓
Old State ✅
```

Therefore:

> **Memento gives us undo/rollback capability without losing previous states.**

🟩 Real-World Use Cases

Your notes mention four major use cases:

1️⃣ Database Transaction Management

```
Transaction
    ↓
Memento
    ↓
Commit / Rollback
```

If transaction fails → rollback.

2️⃣ Version Control Systems

Examples conceptually include:

```
Version 1
   ↓
Version 2
   ↓
Version 3
```

We can return to an earlier version.

3️⃣ Applications with Undo Capability

For example:

```
Type
 ↓
Edit
 ↓
Delete
 ↓
Undo
```

Previous state can be restored.

4️⃣ Failure Recovery

Before performing risky operations:

```
Current State
     ↓
Save Snapshot
     ↓
Perform Operation
     ↓
Failure?
     ↓
Restore Snapshot
```

These are the real-world use cases listed on page 3.

🟩 Important Terms

Term	Meaning
Originator	Object whose state changes
Memento	Snapshot of Originator's state
Caretaker	Manages/stores Mementos
Snapshot	Saved state at a particular point
Rollback	Restore previous state
Commit	Confirm changes permanently
Undo	Go back to previous state

🟩 Easy Example

Imagine a text editor:

```
"Hello"
   ↓
"Hello World"
   ↓
"Hello World!!!"
```

Before changing the text, we save snapshots:

```
M1 → "Hello"
M2 → "Hello World"
M3 → "Hello World!!!"
```

If user presses Undo:

```
Current
"Hello World!!!"

      ↓ undo

"Hello World"
```

Again Undo:

```
"Hello"
```

So:

```
Text Editor = Originator
Snapshots   = Memento
Undo History = Caretaker
```

🟩 Advantages

✅ Undo Support

Previous state restore kar sakte hain.

✅ Rollback

Failed operation ke baad previous consistent state par return kar sakte hain.

✅ State History

Multiple snapshots maintain kiye ja sakte hain.

✅ Encapsulation

Object ki state ko safely snapshot ke form mein maintain kiya ja sakta hai.

✅ Useful for Recovery

Failure hone par saved state restore ki ja sakti hai.

🟩 19. Disadvantages

❌ Memory Usage

Agar bahut saare snapshots store karenge:

```
M1
M2
M3
...
M1000
```

to memory usage increase ho sakta hai.

❌ Large Objects

Agar object's state bahut large hai, snapshot banana expensive ho sakta hai.

❌ History Management

Caretaker ko old snapshots manage/delete karne padte hain.

🟩 Interview Definition ⭐

> **Memento Design Pattern provides the ability to capture and save an object's state at a particular point in time and restore it later without exposing the object's internal details.**

Easy language:

> **Memento = Save State + Restore Previous State**

🟩 Interview Questions

Q1. What is Memento Pattern?

It is a behavioral design pattern used to save an object's state as a snapshot and restore it later.

Q2. What are the three components?

```
Originator
Memento
Caretaker
```

Q3. What does Originator do?

It creates the Memento and restores its state from a Memento.

Q4. What does Memento do?

It stores the snapshot of Originator's state.

Q5. What does Caretaker do?

It manages and stores Mementos/history.

Q6. Where is Memento commonly used?

- Database transactions
- Undo/Redo
- Version control
- Failure recovery